Project Report: VoIP Protocol Analysis Tool using Scapy

Dr. Suman Kumar
Associate Professor and Department Chair
Troy University

Author
Md Azmal Hossain Sarker [ID: 1700875]
Md Mushfiqur Rahman [ID: 1698929]
Tanaka Sumba [ID: 1508769]

May 13, 2025

Contents

1 Introduction **2**
- 1.1 Project Overview . . . 2
- 1.2 Objectives . . . 2
- 1.3 Scope . . . 2

2 Project Architecture **3**
- 2.1 High-Level Overview . . . 3
- 2.2 Module Descriptions . . . 3

3 Implementation Details **4**
- 3.1 Directory Structure . . . 4
- 3.2 Packet Capture (`capture/packet_sniffer.py`, `main.py`) . . . 5
- 3.3 Protocol Decoding (`decoder/sip_decoder.py`, `decoder/rtp_decoder.py`) 6
- 3.4 Metadata Extraction (`extractor/meta_extractor.py`) . . . 7
- 3.5 Traffic Analysis (`analysis/`) . . . 8
- 3.6 Reporting (`reporting/report_generator.py`) . . . 13
- 3.7 Orchestration (`run_analysis.py`, `run_voip_tool.bat`) . . . 14

4 VoIP Protocols Explained **17**
- 4.1 What is VoIP? . . . 17
- 4.2 SIP (Session Initiation Protocol) . . . 17
- 4.3 RTP (Real-time Transport Protocol) . . . 18
- 4.4 Real-World Examples in IT Industries . . . 18

5 How to Run the Tool **18**
- 5.1 Prerequisites . . . 18
- 5.2 Setup . . . 18
- 5.3 Execution . . . 19

6 Conclusion and Future Work **19**
- 6.1 Summary . . . 19
- 6.2 Potential Enhancements . . . 19

1 Introduction

1.1 Project Overview

Voice over Internet Protocol (VoIP) has become a standard for voice communication in modern networks. While offering flexibility and cost savings, it also introduces unique security challenges and performance considerations. This project focuses on developing a command-line tool using the powerful Python library Scapy to capture, decode, and analyze VoIP network traffic. The tool aims to provide insights into VoIP communications, identify potential quality issues, and detect security vulnerabilities or anomalies.

1.2 Objectives

The primary objectives of this project are:

1. **Capture VoIP Traffic:** Intercept VoIP-related network packets (primarily UDP/TCP on SIP ports and RTP port ranges) using Scapy.
2. **Decode VoIP Protocols:** Interpret the structure and content of captured SIP (Session Initiation Protocol) and RTP (Real-time Transport Protocol) packets.
3. **Extract VoIP Metadata:** Systematically pull relevant information from packets, such as IP addresses, port numbers, SIP methods, call identifiers, RTP timestamps, sequence numbers, SSRC identifiers, and potential codec information.
4. **Analyze VoIP Traffic:** Apply analysis techniques to the extracted data to detect patterns, anomalies (e.g., using Isolation Forest), quality issues (jitter, packet loss in RTP streams), and potential security threats (e.g., SIP call floods).
5. **Generate Reports:** Create structured, human-readable reports (initially designed for PDF via FPDF, adapted here for LaTeX) summarizing the analysis findings, including traffic summaries, identified threats, RTP stream statistics, and actionable recommendations.

1.3 Scope

In Scope:

- Capturing packets on a selected network interface based on common VoIP ports (e.g., 5060 for SIP) and UDP traffic (often used by RTP).
- Basic decoding and metadata extraction for SIP messages (INVITE, BYE, ACK, etc.) and RTP headers.
- Analysis of captured *offline* pcap files.
- Detection of basic RTP quality metrics (packet loss, jitter) and simple SIP-based threats (high packet rates, potential call floods).
- Generation of a summary report (this LaTeX document).
- Execution via a batch script on Windows.

Out of Scope:

- Real-time, continuous monitoring and analysis.
- Deep inspection of encrypted VoIP traffic (e.g., SRTP, SIP over TLS) without decryption keys.
- Advanced security threat analysis (e.g., fuzzing, vulnerability scanning).
- Support for less common VoIP protocols or codecs beyond basic identification heuristics.
- A graphical user interface (GUI), although a placeholder for launching one is in the batch script.

2 Project Architecture

2.1 High-Level Overview

The tool follows a sequential workflow, orchestrated primarily by the `run_voip_tool.bat` script. The major steps involve capturing traffic, saving it, then analyzing the saved file to extract metadata, perform RTP and threat analysis, and finally generate a report.

Flow Description:

1. The batch script sets up the environment.
2. `main.py` captures VoIP packets based on a filter and saves them to a `.pcap` file.
3. `run_analysis.py` reads the saved `.pcap` file.
4. Each packet is processed by `extractor/meta_extractor.py` to get basic info.
5. All packets are processed by `analysis/rtp_analysis.py` to calculate RTP stream statistics.
6. The extracted metadata is processed by `analysis/threat_analysis.py` to identify basic SIP threats.
7. Finally, the collected data is formatted into a report (originally PDF via `reporting/report_generato` now this LaTeX document).

2.2 Module Descriptions

- **Capture Module (`main.py`, `capture/packet_sniffer.py`):** Uses Scapy's `sniff` function to listen on a network interface and capture packets matching the VoIP filter. Saves captured packets to a file. `main.py` is the primary script used for capture in the provided `run_voip_tool.bat`.
- **Decoder Module (`decoder/sip_decoder.py`, `decoder/rtp_decoder.py`):** Contains basic functions to parse the payload of potential SIP or RTP packets. Main analysis relies more on other modules.
- **Metadata Extraction Module (`extractor/meta_extractor.py`):** Extracts key fields (IPs, ports) and identifies SIP messages, extracting a summary line.
- **Traffic Analysis Module (`analysis/`)**
  - `rtp_analysis.py`: Calculates RTP stream statistics (loss, jitter, codec guess) by parsing RTP headers directly from the UDP payload.

  - `threat_analysis.py`: Performs simple rule-based threat detection on SIP metadata (high packet rates, call floods).
  - `anomaly_detector.py`: Implements Isolation Forest for anomaly detection (not currently integrated into the main analysis flow).

- **Reporting Module (`reporting/report_generator.py`):** Originally used FPDF for PDF reports. The logic is adapted here for presentation in LaTeX.

- **Orchestration (`run_analysis.py`, `run_voip_tool.bat`):** Scripts to manage the execution flow, handle dependencies, run capture, trigger analysis, and generate the report.

3 Implementation Details

3.1 Directory Structure

Based on the imports and file paths in the code, the project likely follows this structure:

```
voip-analysis-tool/

 venv/                  # Virtual environment
 analysis/
    __init__.py
    anomaly_detector.py
    rtp_analysis.py
    threat_analysis.py
 capture/
    __init__.py
    packet_sniffer.py    # Alternative/module capture script
 data/
    captures/            # Default location for .pcap files
       voip_traffic.pcap (Example)
    logs/                # Default location for reports (PDFs originally)
        voip_traffic_report.pdf (Example)
 decoder/
    __init__.py
    rtp_decoder.py
    sip_decoder.py
 extractor/
    __init__.py
    metadata_extractor.py
 reporting/
    __init__.py
    report_generator.py  # Generates PDF reports
 ui/                    # Placeholder for potential UI
    dashboard.py       # Placeholder dashboard script

 main.py                # Main capture script (used by batch file)
 run_analysis.py        # Script to analyze captured files
 run_voip_tool.bat      # Windows batch script to run the tool
 requirements.txt       # Python package dependencies
```

3.2 Packet Capture (`capture/packet_sniffer.py`, `main.py`)

`capture/packet_sniffer.py`

Purpose: Provides functions to select an interface and capture VoIP traffic. Appears modular but `main.py` is used by the batch script.
Explanation: `select_interface()` lists interfaces and prompts user selection. `capture_voip_packets` uses the selected interface, defines a BPF filter (`"udp or tcp port 5060"`), sniffs traffic for 60 seconds using `scapy.all.sniff()`, and saves packets to `data/captures/voip_traffic.pcap` using `wrpcap()`.

```python
# (Code for packet_sniffer.py as provided in the prompt)
from scapy.all import sniff, wrpcap, get_if_list
import os

def select_interface():
    interfaces = get_if_list()
    print("Available Interfaces:")
    for idx, iface in enumerate(interfaces):
        print(f"{idx + 1}: {iface}")

    choice = input("\nSelect the interface number to capture VoIP packets (e.g., 5): ")
    try:
        iface_index = int(choice) - 1
        if 0 <= iface_index < len(interfaces):
            return interfaces[iface_index]
        else:
            print("[!] Invalid selection. Defaulting to first interface.")
            return interfaces[0]
    except ValueError:
        print("[!] Invalid input. Defaulting to first interface.")
        return interfaces[0]

def capture_voip_packets():
    iface = select_interface()
    bpf_filter = "udp or tcp port 5060"
    output_file = "data/captures/voip_traffic.pcap"
    os.makedirs(os.path.dirname(output_file), exist_ok=True)

    print(f"\n[*] Capturing packets on {iface} with filter: {bpf_filter}")
    try:
        packets = sniff(iface=iface, filter=bpf_filter, timeout=60)
        wrpcap(output_file, packets)
        print(f"[   ] Packets saved to {output_file}")
    except Exception as e:
        print(f"[!] Failed to capture packets: {e}")

if __name__ == "__main__":
    print("[*] Script started.")
    capture_voip_packets()
```

Listing 1: capture/packet_sniffer.py

main.py

Purpose: Main entry point for packet capture invoked by `run_voip_tool.bat`. Attempts automatic interface selection.
Explanation: `get_default_interface()` tries to find the first active network interface. `capture_voip_packets()` calls this, uses the same BPF filter and output file, sniffs, and saves packets.

```python
# (Code for main.py as provided in the prompt)
from scapy.all import sniff, wrpcap, get_if_list, get_if_addr, conf
import os

def get_default_interface():
    interfaces = get_if_list()
    for iface in interfaces:
        try:
            ip = get_if_addr(iface)
            if ip != "0.0.0.0":
                print(f"[*] Selected interface: {iface} (IP: {ip})")
                return iface
        except Exception:
            continue
    raise RuntimeError("No suitable network interface found.")

def capture_voip_packets():
    iface = get_default_interface()
    bpf_filter = "udp or tcp port 5060"
    output_file = "data/captures/voip_traffic.pcap"
    os.makedirs(os.path.dirname(output_file), exist_ok=True)

    print(f"[*] Capturing packets on {iface} with filter: {bpf_filter}")
    try:
        packets = sniff(iface=iface, filter=bpf_filter, timeout=60)
        wrpcap(output_file, packets)
        print(f"[   ] Packets saved to {output_file}")
    except Exception as e:
        print(f"[!] Failed to capture packets: {e}")

if __name__ == "__main__":
    capture_voip_packets()
```

Listing 2: main.py

3.3 Protocol Decoding (`decoder/sip_decoder.py`, `decoder/rtp_decoder.py`)

decoder/rtp_decoder.py

Purpose: Basic heuristic to identify potential RTP packets based on UDP destination port range (>= 16384).
Explanation: Prints source/destination ports if the heuristic matches. Does not decode RTP header details here; more detailed parsing occurs in `rtp_analysis.py`.

```python
# (Code for rtp_decoder.py as provided in the prompt)
def parse_rtp(packet):
    if packet.haslayer('UDP'):
        udp = packet['UDP']
        if udp.dport >= 16384:
            print(f"[RTP?] Src Port: {udp.sport}, Dst Port: {udp.dport}")
            # Heuristics for RTP
```

Listing 3: decoder/rtp_decoder.py

decoder/sip_decoder.py

Purpose: Identifies potential SIP packets in the raw payload and prints key header lines.
Explanation: Checks raw payload for "SIP" or "INVITE". If found, splits into lines and prints common headers (`From:`, `To:`, etc.). Does not provide structured output.

```python
# (Code for sip_decoder.py as provided in the prompt)
def parse_sip_packet(packet):
    if packet.haslayer('Raw'):
        # Ensure payload is treated as bytes then decoded safely
        try:
            payload_bytes = packet['Raw'].load
            payload_str = payload_bytes.decode('utf-8', errors='ignore')
        except Exception:
            payload_str = "" # Handle potential decoding errors

        if "SIP" in payload_str or "INVITE" in payload_str:
            print("[SIP] Packet found")
            # Use splitlines() for potentially better line ending handling
            lines = payload_str.splitlines()
            for line in lines:
                if line.startswith(("From:", "To:", "Call-ID:", "CSeq:", "Via:")):
                    print(f"   {line}")
```

Listing 4: decoder/sip_decoder.py

3.4 Metadata Extraction (extractor/meta_extractor.py)

Purpose: Extracts essential metadata (IPs, ports, protocol type, SIP summary) from each packet into a dictionary.
Explanation: Populates a dictionary with source/destination IP and UDP/TCP ports. Tries to decode the UDP/TCP payload, checks for SIP keywords, and extracts the first significant SIP line (e.g., INVITE, BYE, status code) as a summary if found. Labels protocol as "SIP", "UDP_Other", "TCP_Other", or handles errors.

```python
# (Code for meta_extractor.py including self-corrections)
def extract_metadata(pkt):
    metadata = {}

    if pkt.haslayer("IP"):
        metadata["src_ip"] = pkt["IP"].src
        metadata["dst_ip"] = pkt["IP"].dst

    if pkt.haslayer("UDP"):
        metadata["src_port"] = pkt["UDP"].sport
        metadata["dst_port"] = pkt["UDP"].dport
    elif pkt.haslayer("TCP"): # Also check TCP ports
        metadata["src_port"] = pkt["TCP"].sport
        metadata["dst_port"] = pkt["TCP"].dport

    try:
        raw_data = ""
        payload_layer = None
        if pkt.haslayer("UDP") and hasattr(pkt["UDP"], "payload"):
            payload_layer = pkt["UDP"]
        elif pkt.haslayer("TCP") and hasattr(pkt["TCP"], "payload"):
             payload_layer = pkt["TCP"]

        if payload_layer:
            raw_data = bytes(payload_layer.payload).decode(errors="ignore")

        if "SIP" in raw_data or "INVITE" in raw_data or "BYE" in raw_data:
```

```
            metadata["protocol"] = "SIP"
            lines = raw_data.splitlines() # Use splitlines
            metadata["sip_summary"] = next((line for line in lines if line.strip() and any(x in line for x in ["INVITE", "BYE", "ACK", "REGISTER", "OPTIONS", "CANCEL", "SIP/"])), "SIP Packet")
        else:
            if pkt.haslayer("UDP"):
                # Could potentially be RTP or other UDP protocol
                 metadata["protocol"] = "UDP_Other"
            elif pkt.haslayer("TCP"):
                 metadata["protocol"] = "TCP_Other"
            else:
                 metadata["protocol"] = "Non_UDP_TCP" # If no UDP/TCP payload found

    except Exception:
        metadata["protocol"] = "UNKNOWN_Error" # Error during processing

    # Add packet timestamp
    if hasattr(pkt, 'time'):
        metadata['timestamp'] = pkt.time

    return metadata
```

Listing 5: extractor/meta_extractor.py

3.5 Traffic Analysis (`analysis/`)

`analysis/anomaly_detector.py`

Purpose: Uses scikit-learn's Isolation Forest for anomaly detection on numerical features.
Explanation: The `detect_anomalies` function takes a 2D array of features, fits an Isolation Forest model, and predicts outliers (-1) vs inliers (1). Includes basic validation and error handling.
Note: This module is not integrated into the main analysis script (`run_analysis.py`).

```
# (Code for anomaly_detector.py including self-corrections)
from sklearn.ensemble import IsolationForest
import numpy as np # Import numpy for shape checks

def detect_anomalies(features):
    """
    Detects anomalies in a numerical feature set using Isolation Forest.
    Args:
        features (numpy.ndarray or similar): A 2D array-like structure
                                             where rows are samples and
                                             columns are numerical features.
    Returns:
        numpy.ndarray: An array of predictions (-1 for anomalies, 1 for inliers).
                       Returns empty list on error or invalid input.
    """
    # Basic validation
    if features is None:
       print("[!] Anomaly detection: No features provided.")
       return []

    # Ensure features is a numpy array for shape checks
    if not isinstance(features, np.ndarray):
        try:
            features = np.array(features)
        except Exception as e:
            print(f"[!] Anomaly detection: Could not convert features to numpy array: {e}")
            return []
```

```python

    if features.size == 0:
        print("[!] Anomaly detection: Features array is empty.")
        return []

    if len(features.shape) == 1:
        # Reshape 1D array to 2D array with one feature per sample
        features = features.reshape(-1, 1)
    elif len(features.shape) != 2:
        print("[!] Anomaly detection: Features must be a 1D or 2D array.")
        return []

    # Check if feature array is empty after potential reshape
    if features.shape[0] == 0:
         print("[!] Anomaly detection: No samples in features array.")
         return []

    # Initialize and fit the model
    clf = IsolationForest(contamination=0.1, random_state=42) # Added random_state
    try:
       clf.fit(features)
       predictions = clf.predict(features)
       return predictions
    except Exception as e:
       print(f"[!] Error during anomaly detection: {e}")
       return [] # Return empty on error
```

Listing 6: analysis/anomaly_detector.py

analysis/rtp_analysis.py

Purpose: Analyzes captured packets to identify RTP streams (by SSRC) and calculate statistics (loss, jitter, codec guess).
Explanation: `is_rtp_packet()` checks UDP payload for RTP characteristics (version 2, header length). `analyze_rtp()` iterates packets, uses `is_rtp_packet()`, extracts RTP header fields (seq, timestamp, SSRC, size) directly from UDP payload bytes. It groups info by SSRC, sorts by sequence number, estimates packet loss (handling basic wrap-around), calculates jitter using RFC 3550 approximation, calculates average payload size, and guesses the codec based on typical sizes (G.711, G.729, etc.). Returns a list of stream summaries.

```python
# (Code for rtp_analysis.py including self-corrections)
from collections import defaultdict
from scapy.all import UDP, IP # Import layers for type hinting/checking
import struct # For handling potential parsing errors

def is_rtp_packet(pkt):
    """Checks if a packet appears to be an RTP packet based on UDP payload."""
    # Check necessary layers exist
    if not pkt or not pkt.haslayer(UDP) or not hasattr(pkt[UDP], "load"):
        return False

    payload = bytes(pkt[UDP].load)
    # RTP header is at least 12 bytes
    if len(payload) >= 12:
        try:
            # Version is bits 0-1 of byte 0
            version = payload[0] >> 6
            # Payload type is bits 0-6 of byte 1
            payload_type = payload[1] & 0x7F
        except IndexError:
            return False # Payload too short error somehow missed length check
```

```python

        # Check for RTP version 2 and valid PT range
        if version == 2 and 0 <= payload_type <= 127:
            return True
    return False

def analyze_rtp(packets):
    """Analyzes a list of packets to extract RTP stream statistics."""
    rtp_info = defaultdict(list)
    for pkt in packets:
        # Ensure packet is not None and has necessary layers
        if pkt and pkt.haslayer(IP) and is_rtp_packet(pkt):
            udp_layer = pkt.getlayer(UDP)
            if not udp_layer or not hasattr(udp_layer, "load"):
                continue # Should be caught by is_rtp_packet, but double-check

            payload = bytes(udp_layer.load)
            if len(payload) < 12: # Ensure header is present
                 continue

            try:
                # Extract RTP header fields
                seq = int.from_bytes(payload[2:4], byteorder="big")
                timestamp = int.from_bytes(payload[4:8], byteorder="big")
                ssrc = int.from_bytes(payload[8:12], byteorder="big")
                size = len(payload) # Includes header + payload
                # Store packet arrival time from Scapy packet object
                pkt_time = pkt.time if hasattr(pkt, 'time') else 0.0
                rtp_info[ssrc].append({"seq": seq, "ts": timestamp, "size": size, "time": pkt_time})
            except (IndexError, struct.error) as e:
                # print(f"[!] Error parsing RTP header for a packet: {e}")  Too verbose
                continue


    rtp_summary = []

    for ssrc, pkt_data in rtp_info.items():
        if not pkt_data: continue # Skip if no data for SSRC

        # Sort by sequence number first for loss calculation
        pkt_data.sort(key=lambda x: x["seq"])
        seq_nums = [p["seq"] for p in pkt_data]
        timestamps = [p["ts"] for p in pkt_data]
        sizes = [p["size"] for p in pkt_data]
        packet_times = [p["time"] for p in pkt_data] # Arrival times

        total_received = len(seq_nums)
        if total_received == 0: continue

        # --- Loss Calculation ---
        min_seq = min(seq_nums)
        max_seq = max(seq_nums)
        # Handle sequence number wrap-around (basic check)
        if max_seq > 60000 and min_seq < 5000 and total_received > 1:
             # Calculate expected packets considering wrap-around
             # Find the break point
             break_index = -1
             for i in range(total_received - 1):
                 if seq_nums[i+1] < seq_nums[i]:
                     break_index = i
                     break
             if break_index != -1:
```

```
                # Expected = (Packets before wrap) + (Packets after wrap)
                expected_packets = (65536 - seq_nums[break_index] + seq_nums[break_index+1]) + (total_received - 1)
            else: # Should not happen if wrap detected, fallback
                expected_packets = max_seq - min_seq + 1
        else:
            expected_packets = max_seq - min_seq + 1

        lost = max(0, expected_packets - total_received)

        # --- Jitter Calculation (RFC 3550 Appendix A.8 approximation) ---
        jitter_val = 0
        if total_received > 1:
            # Sort by arrival time for jitter calculation
            pkt_data.sort(key=lambda x: x["time"])
            # Use estimated clock rate (e.g., 8000 Hz for G.711)
            # This is a guess; real clock rate might differ or come from SDP/RTCP
            clock_rate = 8000.0
            for i in range(1, total_received):
                # Difference in arrival times (D) in seconds
                arrival_diff_sec = pkt_data[i]["time"] - pkt_data[i-1]["time"]
                # Convert to timestamp units using clock rate
                arrival_diff_ts = arrival_diff_sec * clock_rate

                # Difference in RTP timestamps (S)
                # Handle timestamp wrap-around
                timestamp_diff = pkt_data[i]["ts"] - pkt_data[i-1]["ts"]
                if timestamp_diff < - (2**31): # Detect wrap-around backward (unlikely but possible)
                    timestamp_diff += 2**32
                elif timestamp_diff > (2**31): # Detect wrap-around forward
                    timestamp_diff -= 2**32

                # Difference D = | S(i) - S(i-1) - (R(i) - R(i-1)) |
                # Or simplified: J(i) = J(i-1) + (|D(i-1,i)| - J(i-1))/16
                delta = abs(timestamp_diff - arrival_diff_ts)
                jitter_val += (delta - jitter_val) / 16.0
            jitter_val = int(jitter_val) # Often reported as integer timestamp units

        # --- Payload and Codec Guess ---
        # Average size of payload (total size - standard 12 byte RTP header)
        avg_payload_size = (sum(sizes) / total_received) - 12 if total_received > 0 else 0
        avg_payload_size = max(0, int(avg_payload_size))

        # Basic codec guessing based on typical payload sizes for 20ms frames
        codec = "Unknown"
        if 155 <= avg_payload_size <= 165: # G.711 (PCMU/PCMA) often 160 bytes
            codec = "G.711 (guessed)"
        elif 18 <= avg_payload_size <= 22: # G.729 often ~20 bytes
            codec = "G.729 (guessed)"
        elif 55 <= avg_payload_size <= 65: # G.722 often ~60 bytes (for 20ms frame?)
            codec = "G.722 (guessed)"
        elif 30 <= avg_payload_size <= 34: # Opus often variable, but ~32 might be common mid-range
            codec = "Opus? (variable, guessed)"

        rtp_summary.append({
            "ssrc": hex(ssrc), # Display SSRC in hex often
            "packets": total_received,
            "expected": expected_packets, # Informational
            "lost": lost,
            "loss_perc": f"{(lost / expected_packets * 100):.2f}%" if
```

```
    expected_packets > 0 else "0.00%",
            "jitter": jitter_val, # Using RFC3550 definition (in timestamp units)
            "avg_payload_bytes": avg_payload_size,
            "codec": codec
        })

    return rtp_summary
```

Listing 7: analysis/rtp_analysis.py

analysis/threat_analysis.py

Purpose: Basic rule-based detection of potential SIP threats from metadata.
Explanation: Counts INVITEs, BYEs, and packets per source IP. Flags IPs exceeding a packet threshold (50) as potential flood/scan. Flags scenarios with several INVITEs but no BYEs in the capture as potential call floods or unterminated calls. Returns a list of threat dictionaries.

```
# (Code for threat_analysis.py including self-corrections)
def detect_threats(metadata_list):
    """Analyzes metadata to detect basic SIP threats."""
    threats = []

    invite_count = 0
    bye_count = 0
    ip_activity = {} # Counts packets per source IP

    for md in metadata_list:
        # Ensure metadata has the required keys before accessing
        src_ip = md.get("src_ip")
        if src_ip:
            ip_activity[src_ip] = ip_activity.get(src_ip, 0) + 1

        # Only count SIP messages for INVITE/BYE counts
        protocol = md.get("protocol")
        if protocol == "SIP":
            summary = md.get("sip_summary", "") # Get sip_summary safely
            if "INVITE" in summary:
                invite_count += 1
            if "BYE" in summary:
                 bye_count += 1

    # Threat Rule 1: High Packet Rate from a single source
    high_rate_threshold = 50 # Configurable threshold
    for ip, count in ip_activity.items():
        if count > high_rate_threshold:
            threats.append({
                "type": "Potential SIP Flood/Scan",
                "details": f"Source IP {ip} sent {count} packets (threshold: {
    high_rate_threshold})"
             })

    # Threat Rule 2: Call Flood / Unterminated Calls
    # Check if there are INVITEs but absolutely no BYEs in the capture
    # This is a weak indicator, could be long calls or short capture
    if invite_count > 5 and bye_count == 0: # Require a few INVITEs
        threats.append({
            "type": "Potential Call Flood / Unterminated Calls",
            "details": f"{invite_count} INVITEs seen without any BYE responses in
    this capture."
         })

    # Add more threat rules here (e.g., scanning patterns for OPTIONS/REGISTER)
```

```python

    # If no specific threats were added, add an info message
    if not threats:
        threats.append({"type": "Info", "details": "No specific SIP threats detected based on current rules."})

    return threats
```

Listing 8: analysis/threat_analysis.py

3.6 Reporting (`reporting/report_generator.py`)

Purpose: Originally generated PDF reports using FPDF. The logic is adapted here to describe the content intended for the report.
Explanation: The script collected metadata, threats, and RTP stats. It generated a PDF with sections for SIP Summary (packet counts, message types), RTP Stream Summary (table with SSRC, packets, loss, jitter, codec, payload size), Detected SIP Threats, and general Recommendations. Helper function `clean()` removed non-ASCII characters for FPDF compatibility.

```python
# (Code for report_generator.py - Note: Output is now this LaTeX doc)
from fpdf import FPDF # This was used for the original PDF output
import os
import re
from datetime import datetime

# --- This clean function was for FPDF ---
def clean(text):
    """Removes non-ASCII characters to prevent FPDF errors."""
    return re.sub(r'[^\x00-\x7F]+', '?', str(text))

# --- The core function logic used for PDF generation ---
def generate_pdf_report(metadata_list, threats, rtp_stats, capture_file="N/A", output_file='data/logs/voip_report.pdf'):
    """Generates a PDF summary report of the VoIP analysis."""
    # ... (Initialization of FPDF object) ...
    # ... (Writing Title, Timestamp, Analyzed File) ...

    # --- Section 1: SIP Traffic Summary ---
    # Calculate total packets, SIP packets
    # Count SIP message types (INVITE, BYE, OK etc.) from metadata["sip_summary"]
    # Write these counts to PDF

    # --- Section 2: RTP Stream Summary ---
    # Create a table header in PDF (SSRC, Packets, Lost, Jitter, Codec, Avg Payload)
    # For each stat in rtp_stats:
    #   Write a row in the PDF table with cleaned data

    # --- Section 3: Detected SIP Threats ---
    # For each threat in threats (excluding "Info" type if others exist):
    #   Write Threat Type and Details to PDF

    # --- Section 4: Recommendations ---
    # Write general recommendation text to PDF

    # ... (Saving the PDF using pdf.output()) ...
    # ... (Error handling for saving) ...
    pass # Function defined for structure, actual PDF generation removed for LaTeX version
```

Listing 9: reporting/report_generator.py (Illustrative - Output adapted to LaTeX)

3.7 Orchestration (`run_analysis.py`, `run_voip_tool.bat`)

`run_analysis.py`

Purpose: Coordinates the analysis phase: reads pcap, calls extractors/analyzers, and triggers report generation.

Explanation: `analyze_pcap()` takes a pcap file path, reads it using `rdpcap()`, calls `extract_metadata()`, `detect_threats()`, and `analyze_rtp()`. It then originally called `generate_pdf_report()`. The `main()` function finds `.pcap` files in `data/captures` and calls `analyze_pcap()` for each. Includes progress messages and error handling.

```python
# (Code for run_analysis.py including self-corrections)
import os
from scapy.all import rdpcap, Scapy_Exception
from extractor.meta_extractor import extract_metadata
from analysis.threat_analysis import detect_threats
from analysis.rtp_analysis import analyze_rtp
# Import removed for LaTeX version: from reporting.report_generator import generate_pdf_report

def analyze_pcap(pcap_path): # Removed output_pdf_path argument
    """Reads a pcap file, performs analysis, and prints results (or would generate report)."""
    print(f"\n[*] Analyzing capture file: {os.path.basename(pcap_path)}")
    try:
        packets = rdpcap(pcap_path)
        print(f"    - Read {len(packets)} packets.")
    except (MemoryError, Scapy_Exception, EOFError) as e: # Catch more specific errors
        print(f"[!] Error reading pcap file '{pcap_path}': {type(e).__name__} - {e}")
        if isinstance(e, MemoryError):
            print("[!]   Try using a smaller capture or processing packets iteratively.")
        return None # Indicate failure
    except Exception as e:
         print(f"[!] Unexpected error reading pcap file '{pcap_path}': {e}")
         return None

    if not packets:
        print("    - No packets found in the file. Skipping analysis.")
        return None

    # --- Extraction ---
    print("    - Extracting metadata...")
    # Use a generator expression potentially for memory efficiency if needed later
    metadata = [extract_metadata(pkt) for pkt in packets if pkt]
    print(f"    - Extracted metadata for {len(metadata)} packets.")

    # --- Analysis ---
    print("    - Analyzing for SIP threats...")
    threats = detect_threats(metadata)
    print(f"    - Found {len([t for t in threats if t.get('type') != 'Info'])} potential SIP threats.")

    print("    - Analyzing RTP streams...")
    rtp_stats = analyze_rtp(packets)
    print(f"    - Found {len(rtp_stats)} potential RTP streams.")

    # --- Reporting (Original Step) ---
    # print("    - Generating PDF report...")
    # generate_pdf_report(metadata, threats, rtp_stats, capture_file=pcap_path, output_file=output_pdf_path)
```

```python
    print("    - Analysis complete. Data available (metadata, threats, rtp_stats).")
    # For LaTeX version, we just return the collected data
    return {"metadata": metadata, "threats": threats, "rtp_stats": rtp_stats}


def main():
    """Main function to find and analyze pcap files."""
    pcap_folder = "data/captures"
    # Output folder no longer needed for direct report generation here
    # output_folder = "data/logs"

    if not os.path.isdir(pcap_folder):
       print(f"[!] Capture folder not found: {pcap_folder}")
       print("[!] Please ensure packets are captured first.")
       return

    analysis_performed = False
    all_results = {} # Store results per file
    for file in os.listdir(pcap_folder):
        if file.lower().endswith((".pcap", ".pcapng")): # Check tuple for endings
            pcap_path = os.path.join(pcap_folder, file)
            # report_filename = f"{os.path.splitext(file)[0]}_report.pdf"  Original PDF name
            # output_pdf = os.path.join(output_folder, report_filename)
            results = analyze_pcap(pcap_path) # Pass path only
            if results:
                all_results[file] = results
                analysis_performed = True

    if not analysis_performed:
       print(f"\n[!] No .pcap or .pcapng files found and analyzed in '{pcap_folder}'.")
    else:
        # Example: Print summary of results after all files analyzed
        print("\n[*] Consolidated Analysis Summary:")
        for filename, data in all_results.items():
            print(f"  --- File: {filename} ---")
            print(f"    SIP Threats Found: {len([t for t in data['threats'] if t.get('type') != 'Info'])}")
            print(f"    RTP Streams Found: {len(data['rtp_stats'])}")


    print("\n[   ] Analysis process complete for all found capture files.")

if __name__ == "__main__":
    main()
```

Listing 10: run_analysis.py

run_voip_tool.bat

Purpose: Windows batch script to automate setup, capture, and analysis.
Explanation: Changes to the script directory, activates Python virtual environment (`venv`), installs dependencies from `requirements.txt`, runs `main.py` for capture, then runs `run_analysis.py` for analysis. Includes error checking at each step and an optional attempt to launch a dashboard UI script.

```bat
@echo off
SETLOCAL ENABLEDELAYEDEXPANSION

REM Get the directory where the batch script is located
SET SCRIPT_DIR=%~dp0
```

```
echo [*] Script Directory: %SCRIPT_DIR%

REM Navigate to the project directory (where the script resides)
cd /d "%SCRIPT_DIR%"
IF %ERRORLEVEL% NEQ 0 (
    echo [!] Failed to change directory to %SCRIPT_DIR%.
    pause
    exit /b 1
)
echo [*] Current Directory: %CD%


REM --- Check for Python Virtual Environment ---
SET VENV_PATH=%SCRIPT_DIR%venv
IF NOT EXIST "%VENV_PATH%\Scripts\activate.bat" (
    echo [!] Virtual environment not found at %VENV_PATH%.
    echo [!] Please create a virtual environment first:
    echo [!]   python -m venv venv
    pause
    exit /b 1
)

REM --- Activate virtual environment ---
echo [*] Activating virtual environment...
call "%VENV_PATH%\Scripts\activate.bat"
IF %ERRORLEVEL% NEQ 0 (
    echo [!] Failed to activate virtual environment.
    pause
    exit /b 1
)
echo [   ] Virtual environment activated.


REM --- Check for requirements.txt ---
IF NOT EXIST "%SCRIPT_DIR%requirements.txt" (
    echo [!] requirements.txt not found in %SCRIPT_DIR%. Cannot install dependencies.
    pause
    exit /b 1
)

REM --- Install required packages ---
echo [*] Installing/Verifying Python dependencies from requirements.txt...
pip install -r requirements.txt
IF %ERRORLEVEL% NEQ 0 (
    echo [!] Package installation failed. Check requirements.txt and network connection.
    pause
    exit /b 1
)
echo [   ] Dependencies are up to date.


REM --- Check for main capture script ---
IF NOT EXIST "%SCRIPT_DIR%main.py" (
    echo [!] main.py script not found in %SCRIPT_DIR%. Cannot start capture.
    pause
    exit /b 1
)

REM --- Run main capture process ---
echo [*] Starting packet capture (main.py)...
python main.py
```

```
IF %ERRORLEVEL% NEQ 0 (
    echo [!] main.py encountered an error during packet capture.
    pause
    exit /b 1
)
echo [   ] Packet capture finished.


REM --- Check for analysis script ---
IF NOT EXIST "%SCRIPT_DIR%run_analysis.py" (
    echo [!] run_analysis.py not found. Skipping analysis step.
) ELSE (
    REM --- Run analysis and report generation ---
    echo [*] Running traffic analysis (run_analysis.py)...
    python run_analysis.py
    IF !ERRORLEVEL! NEQ 0 (
        echo [!] run_analysis.py failed during analysis.
        pause
        exit /b 1
    )
    echo [   ] Analysis finished. Check console output for summaries.
)


REM --- Check for and run the dashboard (Optional) ---
IF EXIST "%SCRIPT_DIR%ui\dashboard.py" (
    echo [*] Launching optional dashboard (ui\dashboard.py)...
    start "VoIP Analysis Dashboard" python ui\dashboard.py
) ELSE (
    echo [*] Optional dashboard file (ui\dashboard.py) not found. Skipping UI launch
    .
)

echo.
echo [   ] All steps completed successfully.
echo [   ] Check the 'data\captures' folder for the PCAP file(s).
echo [   ] Review console output from run_analysis.py for results.
pause
ENDLOCAL
exit /b 0
```

Listing 11: run_voip_tool.bat

4 VoIP Protocols Explained

4.1 What is VoIP?

VoIP stands for Voice over Internet Protocol. It's a technology that allows you to make voice calls using an internet connection instead of a traditional analog phone line. It converts your voice into digital signals (packets) that travel over the internet or any IP network.

4.2 SIP (Session Initiation Protocol)

- **Purpose:** SIP is a **signaling** protocol used primarily for controlling multimedia communication sessions, such as voice and video calls over IP networks. Its main functions include call setup, modification, teardown, user location, availability (presence), and capability negotiation.
- **How it works:** SIP uses text-based messages, similar to HTTP. Common SIP methods (commands) include: `INVITE`, `ACK`, `BYE`, `CANCEL`, `REGISTER`, `OPTIONS`.

- **Transport:** Typically runs over UDP (User Datagram Protocol) for speed, but can also run over TCP (Transmission Control Protocol) or TLS (Transport Layer Security) for reliability and security. Default ports are 5060 (UDP/TCP) and 5061 (TLS).

4.3 RTP (Real-time Transport Protocol)

- **Purpose:** RTP is the protocol responsible for **delivering the actual audio and video data (media streams)** once a call has been established by SIP. It provides features suitable for real-time data: payload type identification (codec), sequence numbering (loss detection, reordering), timestamping (playback timing, jitter calculation), and source identification (SSRC).
- **How it works:** RTP itself doesn't guarantee delivery (it typically runs over UDP). It provides the information needed by the receiving application to handle timing and loss.
- **RTCP (RTP Control Protocol):** Almost always used with RTP. RTCP packets monitor session quality (loss, jitter, round-trip time) and convey participant information.
- **Transport:** Almost exclusively uses UDP, typically on dynamically allocated even-numbered ports (RTCP uses the next higher odd port). Common port ranges are often 16384-32767.

4.4 Real-World Examples in IT Industries

- **Enterprise Communications:** IP PBXs (Cisco, Avaya, FreePBX), cloud services (RingCentral, Vonage), SIP trunks.
- **Contact Centers:** Agent calls, call routing, recording (tapping RTP), CRM integration.
- **Collaboration Platforms:** Microsoft Teams, Zoom, Cisco Webex, Google Meet use SIP (or variants) and RTP/SRTP.
- **Telecommunication Providers:** Core network routing, residential/business VoIP services.
- **Managed Service Providers (MSPs):** Troubleshooting call quality (RTP analysis) and security (SIP analysis) for clients.

5 How to Run the Tool

5.1 Prerequisites

1. **Python:** Python 3.7+ installed and added to system PATH.
2. **Pip:** Python's package installer.
3. **Npcap/WinPcap:** Packet capture driver for Windows (Npcap recommended). Install with WinPcap API-compatible mode if needed (https://npcap.com/).
4. **Administrator Privileges:** Required for packet capture.

5.2 Setup

1. **Download/Clone Project:** Place files in a directory (e.g., `C:\voip-analysis-tool`).
2. **Create Virtual Environment (Recommended):**

```
cd C:\voip-analysis-tool
python -m venv venv
```

3. **Activate Virtual Environment:**

   ```
   venv\Scripts\activate.bat
   ```

4. **Create `requirements.txt`:** Create this file in the project root:

   ```
   scapy>=2.4.5
   fpdf>=1.7.2  # Required by original report generator, keep if needed elsewhe
   scikit-learn>=1.0.0 # For anomaly_detector.py
   ```

5. **Install Dependencies** (while venv is active):

   ```
   pip install -r requirements.txt
   ```

Note: The `run_voip_tool.bat` script automates steps 3 and 5.

5.3 Execution

1. Open Command Prompt **as Administrator**.
2. Navigate to the project directory (e.g., `cd C:\voip-analysis-tool`).
3. Run the batch script: `run_voip_tool.bat`

The script will activate the environment, install dependencies, run `main.py` to capture traffic (default 60s to `data/captures/voip_traffic.pcap`), and then run `run_analysis.py` to analyze captures in the `data/captures` folder, printing summaries to the console.

6 Conclusion and Future Work

6.1 Summary

This project successfully developed a command-line VoIP Protocol Analysis Tool using Scapy. The tool meets the core objectives of capturing VoIP traffic, performing basic decoding of SIP and RTP, extracting relevant metadata, conducting preliminary analysis for quality metrics (loss, jitter) and simple threats, and generating summary reports (originally PDF, now demonstrated via console output and this LaTeX structure). The modular structure allows for potential expansion, and the batch script provides a convenient execution workflow on Windows.

6.2 Potential Enhancements

- **Real-time Analysis:** Adapt the tool for live traffic analysis.
- **Advanced Decoders:** Implement more robust Scapy layers or parsers for SIP/RTP/SDP/RTCP.
- **Call Flow Reconstruction:** Correlate SIP messages to reconstruct individual call flows.
- **SRTP/TLS Support:** Add capabilities for handling encrypted traffic (e.g., importing keys).

- **Enhanced Threat Detection:** Implement more sophisticated rules (malformed packets, fuzzing detection, etc.).
- **Improved RTP Analysis:** Refine jitter/loss calculations, incorporate RTCP analysis, improve codec detection (e.g., via SDP).
- **Anomaly Detection Integration:** Define relevant numerical features and integrate the `anomaly_detector.py` module into the main flow.
- **Cross-Platform Support:** Provide shell scripts (`.sh`) for Linux/macOS.
- **Configuration File:** Manage settings (filters, thresholds) via a config file.
- **Graphical User Interface (GUI):** Develop a GUI for user-friendly interaction and visualization.